```
import torch, math
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer, TrainingArguments, DataCollatorForLanguageModeling, set_s
from datasets import Dataset

set_seed(42)

tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 1.23kB/s]
vocab.json: 1.04M/? [00:00<00:00, 11.4MB/s]
merges.txt: 456k/? [00:00<00:00, 5.10MB/s]
tokenizer.json: 1.36M/? [00:00<00:00, 9.40MB/s]
config.json: 100%  665/665 [00:00<00:00, 28.4kB/s]
model.safetensors: 100%  548M/548M [00:05<00:00, 119MB/s]
Loading weights: 100%  148/148 [00:00<00:00, 250.84it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100%  124/124 [00:00<00:00, 3.94kB/s]
```

```
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(model.device)

    with torch.no_grad():
        output = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(output[0], skip_special_tokens=True)

review_prompts = [
    "This product is",
    "I bought this phone and",
    "The quality of this item"
]

baseline = {}

print("=== BASELINE OUTPUT ===")
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}\nOutput: {baseline[p]}")
```

```
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, yo
=== BASELINE OUTPUT ===

Prompt: This product is
Output: This product is made from high quality, lightweight stainless steel. If you are looking for something a little more

Laser Pouch

Not all of our laser printers are created equal. We have a laser printer that comes with all of our printer parts. These par
```

```
Prompt: I bought this phone and
Output: I bought this phone and I have not used it on a lot of people. I have also not used it on any other people.

The screen was amazing and the sound was amazing. It was not loud. I would never use it on a tv, laptop, smartphone or other

The battery life is good. The phone works great but it has so many problems.

I have been using phones that have the Snapdragon 616 processor with the 8

Prompt: The quality of this item
Output: The quality of this item in the item description (and if the item is already in stock) will determine how many times

This item is currently out of stock and may be out of stock based on the availability of your specific request.

Please contact us before placing your order to verify availability.

Your order is now up for pre-order.
```

```
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments and coding projects perfectly.",
    "the sound quality of these headphones is incredible with deep bass and clear vocals.",
    "this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.",
    "great wireless earbuds with noise cancellation that blocks out all background sound.",
    "the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.",
    "this portable charger saved me during travel and it charges my phone three times on a single charge.",
    "the tablet screen is bright and colorful which makes watching movies a great experience.",
    "i love this fitness tracker because it motivates me to reach my daily exercise goals.",
    "this bluetooth speaker is compact but delivers surprisingly loud and clear audio.",
    "the delivery was fast and the product was packed securely with no damage at all.",
    "excellent value for money and the build quality feels premium despite the affordable price.",
    "the customer service team was very helpful when i had questions about the product features.",
    "this camera takes stunning photos in low light and the video recording quality is very smooth.",
    "i have been using this product for three months and it still works perfectly like day one.",
    "the design is sleek and modern and it looks great on my desk next to my other gadgets.",
    "easy to set up right out of the box and the instructions were clear and simple to follow.",
    "highly recommend this product to anyone looking for quality and reliability at a fair price.",
    "the software updates keep adding new features which makes this purchase even more worthwhile.",
    "best purchase i made this year and i would definitely buy from this brand again."
]

dataset = Dataset.from_dict({"text": corpus})

tokenized = dataset.map(
    lambda x: tokenizer(x["text"], truncation=True, padding="max_length", max_length=128),
    batched=True,
    remove_columns=["text"]
)

split = tokenized.train_test_split(test_size=0.15)

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./gpt2-reviews",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available()
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=data_collator
)

trainer.train()
```

```
Map: 100%                                                    20/20 [00:00<00:00, 557.33 examples/s]
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
[45/45 00:03, Epoch 5/5]
```

Step	Training Loss
10	3.820944
20	3.000968
30	2.112275
40	1.603487

```
TrainOutput(global_step=45, training_loss=2.5480843438042533, metrics={'train_runtime': 4.613, 'train_samples_per_second':
18.426, 'train_steps_per_second': 9.755, 'total_flos': 5552455680000.0, 'train_loss': 2.5480843438042533, 'epoch': 5.0})
```

```
print("\n=== AFTER FINE-TUNING ===")

for p in review_prompts:
    output = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}")
    print(f"Before: {baseline[p][:100]}")
    print(f"After : {output[:100]}")
```

```
=== AFTER FINE-TUNING ===

Prompt: This product is
Before: This product is made from high quality, lightweight stainless steel. If you are looking for somethin
After : This product is designed to work with all of the Samsung Galaxy Note 7 models and all of the Galaxy

Prompt: I bought this phone and
Before: I bought this phone and I have not used it on a lot of people. I have also not used it on any other
After : I bought this phone and love the experience of using it with the included earbuds. The phone feels m

Prompt: The quality of this item
Before: The quality of this item in the item description (and if the item is already in stock) will determin
After : The quality of this item is excellent. The draw was very clear and the draw thread was very smooth.
```

```
tokenizer2 = GPT2Tokenizer.from_pretrained('gpt2')
model2 = GPT2LMHeadModel.from_pretrained('gpt2')

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id
```

```
Loading weights: 100%                                    148/148 [00:00<00:00, 630.35it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
recipe_prompts = [
    "To make butter chicken",
    "For pasta carbonara",
    "To prepare a chocolate cake"
]

baseline2 = {}

print("=== BASELINE RECIPES ===")
for p in recipe_prompts:
    baseline2[p] = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}\nOutput: {baseline2[p]}")
```

```
=== BASELINE RECIPES ===

Prompt: To make butter chicken
Output: To make butter chicken, or chicken, or chicken breasts are only one possible method of making it. To cook with butte

Heat one tablespoon of butter over medium heat. If the butter is too hot, then it is going to get soggy. You can also heat t

Add the sugar and stir vigorously until the butter

Prompt: For pasta carbonara
Output: For pasta carbonara are usually in a very low gravity. The Italian term is "spaghetti carbonara".

A very thin pasta carbonara is a nice, soft pasta that is a regular pasta. However, the carbonara will not be able to cover

So
```

```
Prompt: To prepare a chocolate cake
Output: To prepare a chocolate cake, I made a couple of different types of cake for myself. You have to be very careful, bec

There is an easy recipe for that on the blog. I also made one of those waffles on my family's birthday. I love them,
```

```
recipes = [
    "to make butter chicken start by marinating chicken pieces in yogurt with spices.",
    "heat butter and fry onions until golden brown then add ginger garlic paste.",
    "add tomato puree and cook until oil separates.",
    "add chicken and cook until fully done.",
    "finish with cream and serve hot.",
    "for pasta carbonara boil pasta until al dente.",
    "fry pancetta until crispy.",
    "mix egg yolk cheese and pepper.",
    "combine pasta with mixture and serve.",
    "to prepare vegetable stir fry heat oil in a wok.",
    "add vegetables and stir fry for three minutes.",
    "add sauces and cook briefly.",
    "serve hot with rice.",
    "for cookies mix butter sugar eggs and flour.",
    "add chocolate chips and bake at 180 degrees.",
    "cool before serving."
]

dataset2 = Dataset.from_dict({"text": recipes})

tokenized2 = dataset2.map(
    lambda x: tokenizer2(x["text"], truncation=True, padding="max_length", max_length=128),
    batched=True,
    remove_columns=["text"]
)

split2 = tokenized2.train_test_split(test_size=0.15)

collator2 = DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)

args2 = TrainingArguments(
    output_dir="./gpt2-recipes",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available()
)

trainer2 = Trainer(
    model=model2,
    args=args2,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=collator2
)

trainer2.train()
```

Map: 100% 16/16 [00:00<00:00, 737.04 examples/s]

[35/35 00:02, Epoch 5/5]

Step	Training Loss
10	4.249005
20	3.131000
30	2.006808

```
TrainOutput(global_step=35, training_loss=2.951051548549107, metrics={'train_runtime': 2.4906, 'train_samples_per_second':
26.098, 'train_steps_per_second': 14.053, 'total_flos': 4245995520000.0, 'train_loss': 2.951051548549107, 'epoch': 5.0})
```

```
print("\n=== AFTER FINE-TUNING RECIPES ===")

for p in recipe_prompts:
    output = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}")
    print(f"Before: {baseline2[p][:100]}")
    print(f"After : {output[:100]}")
```

```
=== AFTER FINE-TUNING RECIPES ===

Prompt: To make butter chicken
Before: To make butter chicken, or chicken, or chicken breasts are only one possible method of making it. To
```

```
After : To make butter chicken.

Preheat oven to 350 degrees F. Preheat oven to 180 degrees F.

Salt and pep

Prompt: For pasta carbonara
Before: For pasta carbonara are usually in a very low gravity. The Italian term is "spaghetti carbonara".

A
After : For pasta carbonara chicken pieces marinate chicken pieces in olive oil. Add pasta by hand. Cover ch

Prompt: To prepare a chocolate cake
Before: To prepare a chocolate cake, I made a couple of different types of cake for myself. You have to be v
After : To prepare a chocolate cake pan and brown the butter in a wok. Add the ginger garlic paste. Cook for
```